Programming Project Report 2

Warren Roberts
100450116

Problem Statement:

The goal of this programming assignment was to design and implement an interactive Pac-Man style simulation using C++ and OpenGL. The program required object-oriented programming and design, rendering 2D shapes using OpenGL primitives, and structuring the system using the Model-View-Controller (MVC) architecture. Normal inputs included movements with the keyboard and internal timing updates with variable dt that passes within update and tick methods. The program will allow you to play the game, post some collisions for debugging, and display you have won or game over. A lot of link errors occurred because I did not fully implement everything when I declared it, so I had to go searching for what all I needed to implement. Some pointer issues occurred to, to fix this I made model hold a pointer to pacman that allows it to use pacman methods!

Design:

I decided to design this project based on MVC architecture to have a clean easy to read project. I used vectors to class objects and to unique pointers of classes to allow for collision detection. I also had many global variables, some statics, and more virtual methods to allow for overriding. I also used uniform_real_distribution, uniform_int_distribution, and std::mt19937 to allow for random number generation. Structures for bounds and colors was implemented to allow for random colors on spawn for ghosts and fruit. Random direction selection, random color selection, entity-to-entity collision checking, and time based direction changes were all implemented into algorithms that increased game functionality. It took so long to implement everything correctly that I did not have much time to add any new features (my family visiting right now added to that). But the implementation I did make seem to work very well with no bugs!

Implementation:

I began by creating the skeleton of all classes / objects that I wanted to implement. This involved choosing what classes inherit from who and what functions should be for all to inherit and what functions should be individual to a certain entity. After this I began working on the game engine and updates for all classes. I started using my code from homework one in order to have a good starting point. I restructured polygon to only be 2D shapes and made sure vertex had nothing unnecessary. With this I created the entity class for pacman, ghosts, and fruit to inherit from. I spent everyday for about two weeks working on this project, putting little by little implementations in play.

Computer Graphics Project

AI Usage Tracking Report

Student: Warren Roberts
Course: Computer Graphics
Project: MVC Pacman-style Game
AI Tool Used: ChatGPT

Summary of AI Usage

AI assistance was used strictly for:

- Architectural guidance
- Debugging explanations
- Design refinement
- Best-practice validation

No complete systems were auto generated and pasted without understanding. All implementation decisions were integrated and modified manually.

Logged AI Interactions

---

1. Random Float Generator for Entity Spawning

User Request:
Create an optimized random float generator between -1.0f and 1.0f and determine best MVC placement.

AI Guidance Provided:

- Use `std::mt19937` seeded once in `Model`
- Use `std::uniform_real_distribution<float>`
- Avoid reseeding per call
- Place RNG inside `Model` for architectural correctness

Impact on Project:

- Clean, deterministic entity spawning
- Improved RNG design
- Proper separation of concerns (Model owns randomness)

---

2. Linker Error: Undefined Symbol for `Model::randFloat()`

User Request:
Explain linker error regarding undefined symbol for `Model::randFloat()`.

AI Guidance Provided:

- Clarified difference between declaration and definition
- Explained missing `Model::` scope resolution
- Explained missing `.cpp` compilation in Makefile

Impact on Project:

- Resolved linking error
- Improved understanding of compilation vs linking
- Strengthened C++ symbol knowledge

---

3. Choosing Data Structure for Tuple Storage

User Request:
Best data structure to store list of tuples.

AI Guidance Provided:

- Recommended `std::vector`
- Recommended struct over tuple for clarity
- Explained why `std::list` is poor for game engines

Impact on Project:

- Transitioned to `struct Color`
- Improved code readability
- Better engine-like structure

---

4. Color System + Shuffle Bag Implementation

User Request:
Prevent fruit and ghost from spawning with same color.

AI Guidance Provided:

- Introduced shuffle “color bag” system
- Explained why reseeding RNG caused repeated matches
- Provided vector shuffle + pop strategy

Impact on Project:

- Deterministic fair color selection
- No repeated colors per cycle
- Cleaner game-feel design

---

5. Random Direction Changes via Timer

User Request:
Make fruit and ghost change direction at random intervals.

AI Guidance Provided:

- Recommended time-based system (dt)
- Avoid frame counters
- Add `changeTimer` and `nextChangeTime`
- Recalculate direction via `atan2`

Impact on Project:

- Frame-rate independent behavior
- Professional timer-based AI movement
- Improved realism

---

6. Entity-Based Collision System Design

User Request:
Implement collision system using left/right/top/bottom bounds.

AI Guidance Provided:

- Introduced AABB collision
- Added `Bounds` struct
- Added `bounds()` function in `Entity`
- Explained penetration resolution
- Explained axis-based velocity flipping

Impact on Project:

- Structured collision detection
- Clean object-oriented collision system
- Proper inheritance usage

---

7. Unique Pointer Polymorphism Error

User Request:
Error: `unique_ptr<Entity>` has no member `setDirection`.

AI Guidance Provided:

- Explained polymorphism and base-class visibility
- Clarified need to declare functions in `Entity`
- Correct use of `->` operator
- Warned about object slicing

Impact on Project:

- Correct smart pointer usage
- Proper inheritance structure
- Prevented slicing bugs

---

8. Safe Object Destruction (Fruit on Collision)

User Request:
Safest way to destroy fruit after collision (was calling `~Fruit()`).

AI Guidance Provided:

- Explained why manual destructor calls are unsafe
- Introduced mark-dead pattern
- Implemented erase-remove idiom with `unique_ptr`

Impact on Project:

- Memory safety guaranteed
- No double-destruction risk
- Clean entity lifecycle management

---

9. Power-Up Timer System (5-Second Auto Disable)

User Request:
Auto-disable power state after 5 seconds.

AI Guidance Provided:

- Add `powerTimer`
- Use delta time
- Frame-rate independent countdown
- Reset timer on early disable

Impact on Project:

- Professional timed power-up system
- Clean Pacman state logic

---

10. HUD / Lives Display

User Request:
Display lives, hurt, and power on screen.

AI Guidance Provided:

- Implemented `drawText`
- Converted bool to string via ternary
- Suggested color-coded states

Impact on Project:

- Functional HUD overlay
- Clean status display system

---

11. Game Over System

User Request:
When lives == 0, display GAME OVER before closing.

AI Guidance Provided:

- Add `gameOver` state in Model
- Add `gameOverTimer`
- Stop simulation
- Delay exit via timer

Impact on Project:

- Controlled game shutdown
- Proper state-driven architecture
- Clean user feedback before termination

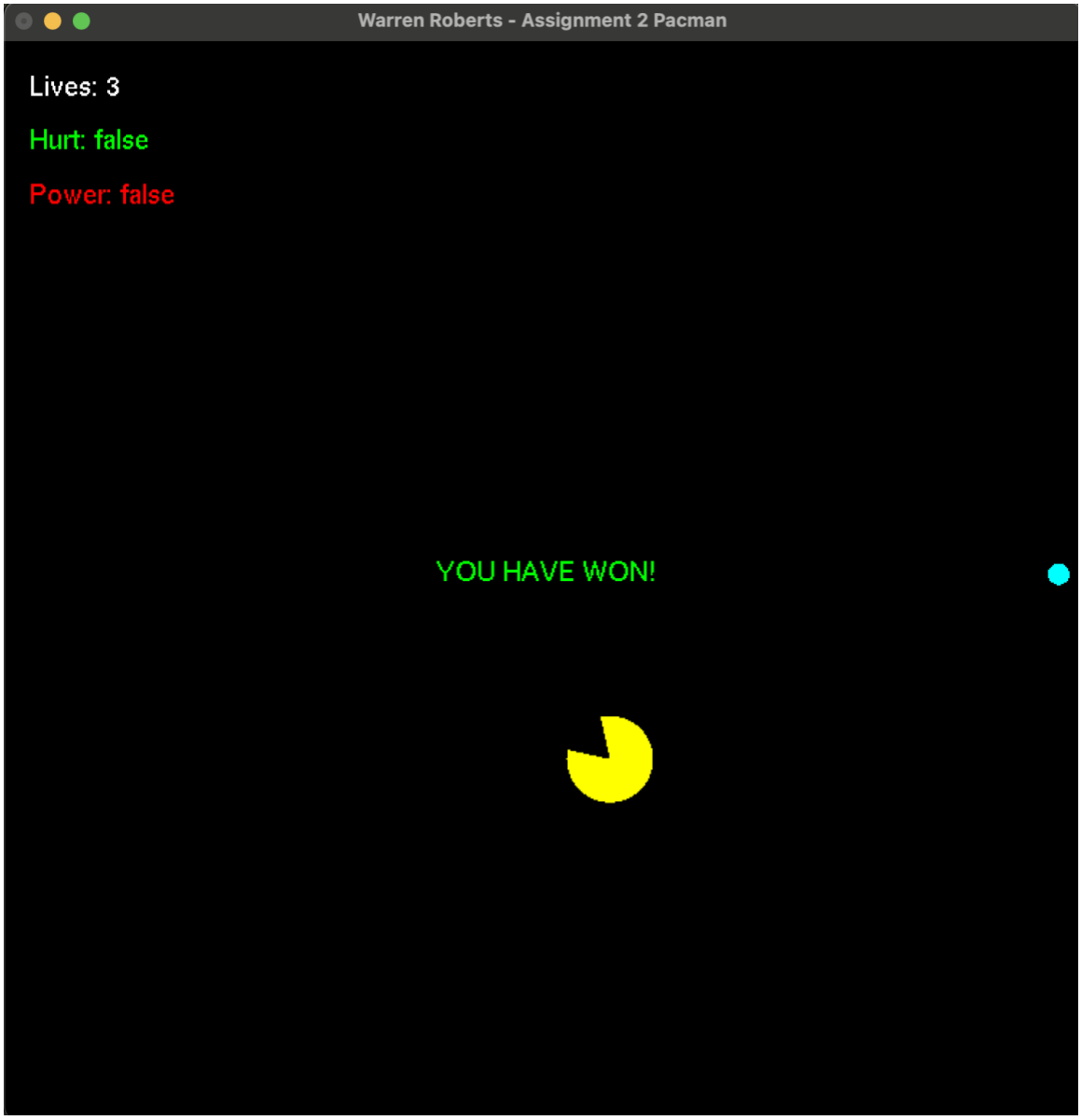
Overall AI Contribution

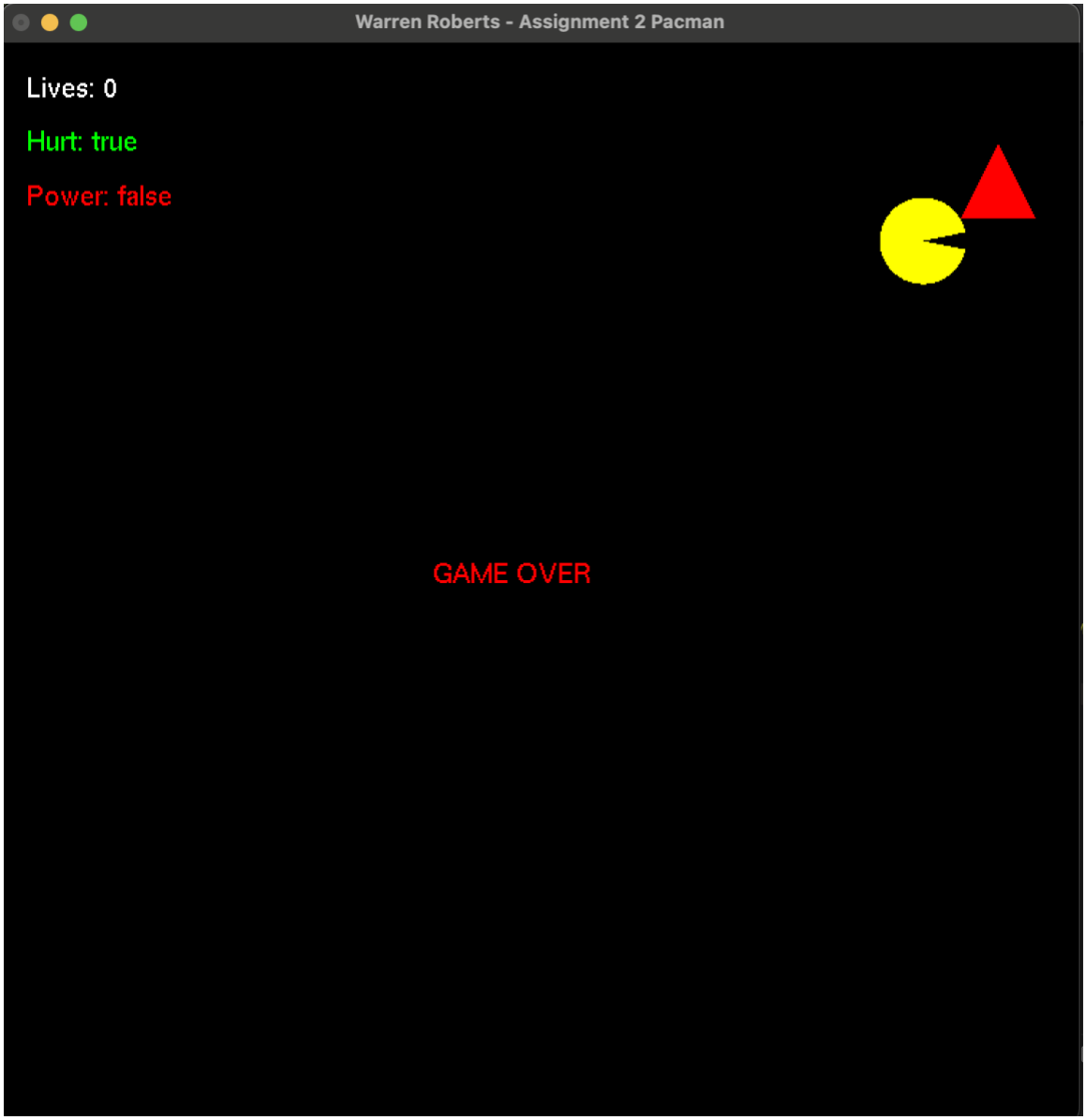
AI assistance was used for:

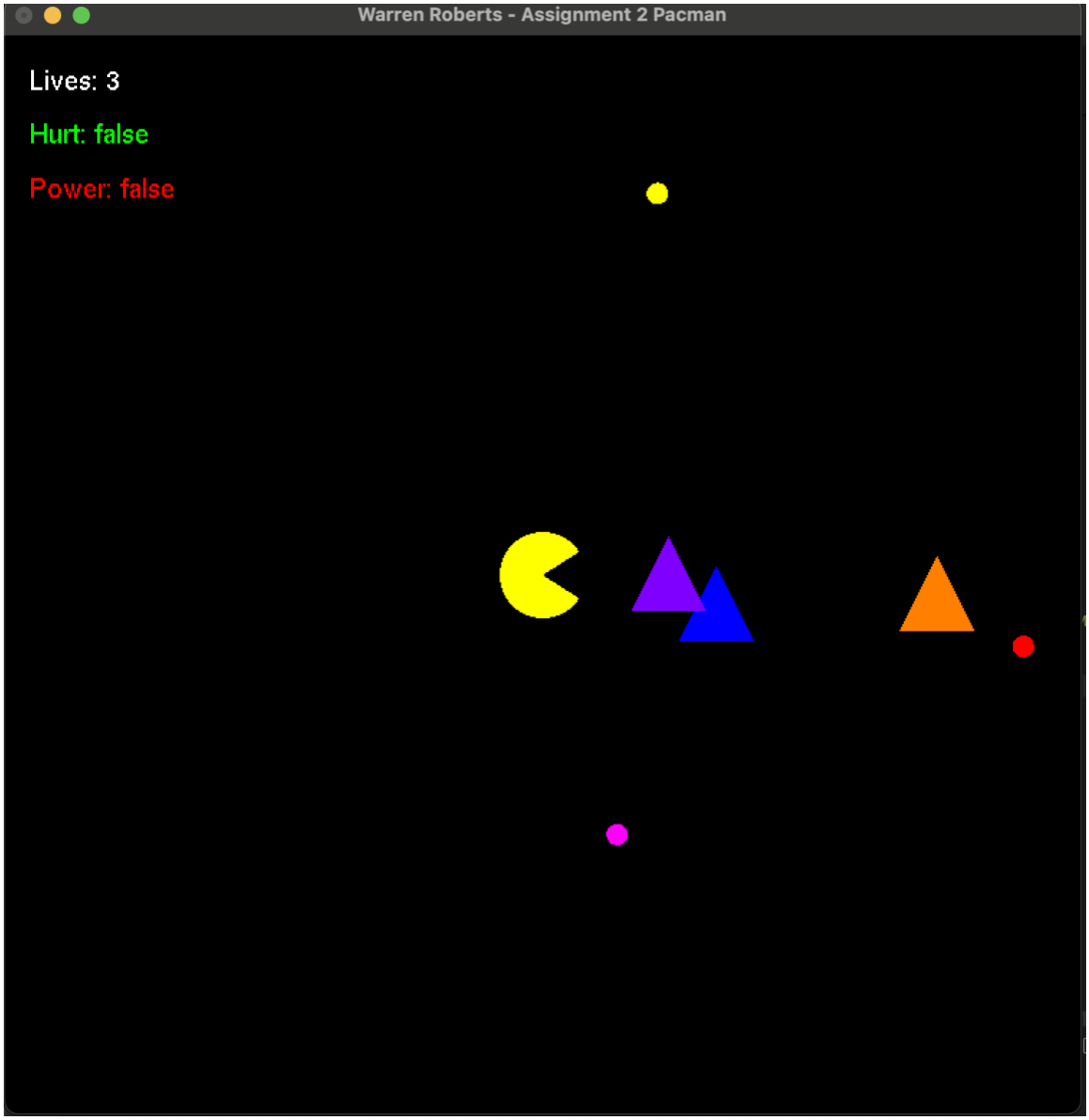
- Debugging explanation
- Architectural design validation
- Collision system structuring
- Memory safety best practices
- Game loop and timer logic

Testing:

I ran the program countless time and live-tested algorithms and their behaviors, comparing their actual and expected behaviors. From this I found many movement bugs, especially with direction. Direction was tricky to implement correctly and I did not even fully do what I wanted (I wanted to make ghosts and fruits bounce off each other, but time did not allow). Normal inputs involved dt to implement time-based changes, as well as keyboard inputs to maneuver throughout the screen. I also added three features to the screen: lives, power status, and hurt status, which will allow you to see if pacman has remaining lives, has an active power, and if he is hurt. I had to test random color duplication and that involved a ton of work to fix! Boundaries were also implemented incorrectly at first using left, right, top, bottom dimensions instead of simply using radius to track size. After fixing this movement and collisions were handled much smoother. Everything worked well after much bug testing and error fixing.


Warren Roberts - Assignment 2 Pacman
Lives: 3
Hurt: false
Power: false
YOU HAVE WON!


Warren Roberts - Assignment 2 Pacman
Lives: 0
Hurt: true
Power: false
GAME OVER



Conclusions:

The final result of this assignment is a fully interactive Pac-Man style game built using OpenGL and an MVC architecture. Pac-Man can be controlled using both keyboard and mouse input, food and ghosts move independently across the screen, and collision detection drives the game logic. The game includes a working power-up system, a hurt cooldown system, randomized ghost behavior, smooth animation, and a three-life system that ends the game when lives reach zero. Overall, the game behaves consistently and meets the functional and interaction requirements outlined in the assignment. This project took around 2 weeks to complete with daily work, if I were to do this again, I would spend more time developing obstacles like walls and a larger map with more things to do and interact with.